

Git & GitHub

A Complete Tutorial for Freshers

Topics Covered

Overview of Git & Git Rebase	Overview of GitHub
• Branching basics • Rebase concepts • Commit history management	• Repositories & Branches • Issues & Pull Requests • Collaboration features
Set Up & Create a Git Repository	Set Up & Configure GitHub Account
• git init command • Configuration settings • Initial project structure	• SSH keys setup • Access tokens • Repository permissions
CRUD Operations on Git	
• add, commit, status, log, push, pull and merge	

*Prepared for: New Developer Onboarding Program
Level: Beginner | Format: Hands-On Tutorial with Examples*

Chapter 1: Overview of Git & Git Rebase

1.1 What is Git?

Git is a distributed version control system (VCS) that helps developers track changes in their source code during software development. Created by Linus Torvalds in 2005, Git is the most widely used version control system in the world today.

Think of Git like a time machine for your code. Every time you save a snapshot (called a "commit"), you can go back in time to any previous version of your project. This is incredibly useful when:

- You accidentally break something and need to go back to a working version
- Multiple developers are working on the same project simultaneously
- You want to try new features without affecting the stable codebase
- You need to track who made what changes and why

Key Concepts in Git

Term	Description
Repository (Repo)	A folder that Git is tracking. Contains all your project files and the history of changes.
Working Directory	The local folder where you see and edit your files.
Staging Area (Index)	A preparation zone where you select which changes to include in the next commit.
Commit	A snapshot of your staged changes saved permanently in the repository history.
Branch	A separate line of development, like a parallel copy of your project.
HEAD	A pointer indicating your current position in the commit history (usually the latest commit on your branch).
Remote	A version of your repository hosted on a server (e.g., GitHub, GitLab).

1.2 Git Workflow - The Big Picture

Understanding the three areas of Git is fundamental before writing any commands:

Note: Files move through three stages: Working Directory → Staging Area → Repository. The git add command moves files to staging, and git commit saves them permanently.

```

Working Directory  -->  git add  -->  Staging Area  -->  git commit  -->
Repository

[Your files]                [Ready to save]                [Saved history]

```

```
Remote Repository <-- git push / git pull --> Local Repository
```

1.3 Branching Basics

A branch in Git is a lightweight movable pointer to a specific commit. The default branch is usually called main (or master in older repositories). Branching allows you to:

- Develop features in isolation without disturbing the main codebase
- Work on bug fixes without interrupting ongoing feature development
- Experiment freely, knowing you can discard the branch if things go wrong
- Enable multiple developers to work independently on different features

Common Branching Commands

```
# List all local branches
git branch

# Create a new branch
git branch feature/login-page

# Switch to an existing branch
git checkout feature/login-page

# Create AND switch to a new branch in one command (modern shortcut)
git checkout -b feature/login-page

# Using the newer 'switch' command (Git 2.23+)
git switch -c feature/login-page

# Merge a branch into the current branch
git merge feature/login-page

# Delete a branch after merging
git branch -d feature/login-page
```

Example: Branching Workflow

Scenario: You are adding a new user registration feature to a project.

1. You are on main branch (stable, working code)
2. Create a new branch for your feature
3. Develop and commit your changes on the feature branch
4. Once done, merge it back into main

```
# Step 1: Make sure you are on main and it's up to date
git checkout main
git pull origin main

# Step 2: Create feature branch
git checkout -b feature/user-registration

# Step 3: Make changes, then commit
# (edit your files...)
git add register.html
git commit -m "Add user registration form"

git add register.js
git commit -m "Add registration form validation logic"

# Step 4: Switch back to main and merge
git checkout main
git merge feature/user-registration

# Step 5: Clean up
git branch -d feature/user-registration
git push origin main
```

1.4 Git Rebase - Managing & Rewriting Commit History

Git rebase is a powerful command used to integrate changes from one branch into another. While git merge creates a new "merge commit" preserving all branch history, git rebase rewrites the commit history to appear as if it happened in a straight, linear line.

Note: Rule of thumb: Never rebase commits that have already been pushed to a shared remote branch. Rebasing rewrites history, which causes problems for others who have the old history.

Merge vs. Rebase - Visual Comparison

```
--- BEFORE (both start from same point) ---
A---B---C   (main)
  \
   D---E   (feature-branch)

--- AFTER git merge ---
A---B---C---M   (main, M = merge commit)
  \       /
   D---E   (feature-branch preserved)

--- AFTER git rebase ---
A---B---C---D'---E'   (main, D' and E' are rewritten commits)
(clean, linear history, no merge commit)
```

Basic Rebase Commands

```
# Basic rebase: reapply commits from feature-branch on top of main
git checkout feature-branch
git rebase main

# Interactive rebase: rewrite the last 3 commits
git rebase -i HEAD~3

# If conflicts arise during rebase, resolve them then:
git rebase --continue

# To cancel the entire rebase operation:
git rebase --abort
```

Interactive Rebase - Rewriting Commit History

Interactive rebase (`git rebase -i`) lets you modify individual commits before replaying them. This is useful for cleaning up messy commits before sharing your work.

```
# Open interactive rebase for last 3 commits
git rebase -i HEAD~3

# Git opens an editor showing your commits like this:
pick a1b2c3d First commit message
pick e4f5g6h Second commit message
pick i7j8k9l Third commit message

# Available commands to replace 'pick':
# pick      = keep this commit as-is
# reword    = keep the commit but edit the commit message
# squash    = combine this commit with the previous one
# fixup     = like squash but discard this commit's message
# drop      = remove this commit entirely

# Example: squash last 3 commits into one clean commit:
pick a1b2c3d Add login feature
squash e4f5g6h Fix typo in login
squash i7j8k9l Fix another typo in login
```

Chapter 2: Set Up and Create a Git Repository

2.1 Installing Git

Before you start, verify that Git is installed on your machine:

```
# Check if Git is installed and see the version
git --version

# Example output:
# git version 2.42.0
```

If Git is not installed, download it from <https://git-scm.com/downloads> and follow the installer for your operating system (Windows, macOS, or Linux).

2.2 Git Configuration Settings

Before creating any repository, configure your identity. Git uses this information to tag every commit you make. This is a one-time setup per machine.

Setting Global Configuration

```
# Set your full name (this appears in commit history)
git config --global user.name "Jane Smith"

# Set your email address (use the same email as your GitHub account)
git config --global user.email "jane.smith@example.com"

# Set your preferred text editor for commit messages
git config --global core.editor "code --wait" # VS Code
# git config --global core.editor "vim"      # Vim
# git config --global core.editor "nano"     # Nano

# Set default branch name to 'main' (modern standard)
git config --global init.defaultBranch main

# Enable colored output in the terminal
git config --global color.ui auto

# View all your current global configuration settings
git config --global --list
```

Configuration Levels Explained

Level	Scope & Location
--system	Applies to ALL users on the computer. Stored in /etc/gitconfig
--global	Applies to ALL repositories for the CURRENT user. Stored in ~/.gitconfig
--local	Applies ONLY to the current repository. Stored in .git/config inside the repo

Note: --local settings override --global settings, which override --system settings.

2.3 Creating a Git Repository with git init

There are two ways to get a Git repository: initialize a brand new one, or clone an existing one from a remote server.

Method 1: Initialize a New Repository

```
# Step 1: Create a project folder and navigate into it
mkdir my-first-project
cd my-first-project

# Step 2: Initialize an empty Git repository
git init

# Output you will see:
# Initialized empty Git repository in /path/to/my-first-project/.git/

# Step 3: Verify the hidden .git folder was created
ls -la      # On macOS/Linux
dir /a      # On Windows

# You should see a .git folder - this is where Git stores everything
```

Method 2: Clone an Existing Repository

```
# Clone a remote repository to your local machine
git clone https://github.com/username/repository-name.git

# Clone into a specific folder name
git clone https://github.com/username/repository-name.git my-local-folder

# Clone a specific branch only
git clone -b develop https://github.com/username/repository-name.git
```

2.4 Initial Project Structure Creation

Once your repository is initialized, set up a good project structure. Here is an example for a web project:

```
# Create a basic web project structure
mkdir src tests docs
touch src/index.html src/styles.css src/app.js
touch README.md

# Your project structure should look like:
my-first-project/
.git/                (hidden - Git internal files, do not touch)
src/
  index.html
  styles.css
  app.js
tests/
docs/
README.md
.gitignore            (we will create this next)
```

Creating a .gitignore File

The .gitignore file tells Git which files and folders to ignore (not track). This is essential for keeping sensitive data and generated files out of version control.

```
# Create a .gitignore file
touch .gitignore

# Open it and add patterns for files to ignore.
# Example .gitignore content for a Node.js project:

# Dependencies folder
node_modules/

# Build output
dist/
build/

# Environment files with secrets
.env
.env.local

# Operating system files
.DS_Store      # macOS
Thumbs.db      # Windows

# IDE/Editor files
.vscode/
.idea/
```



```
# Log files
*.log
npm-debug.log*
```

Creating Your First Commit

```
# Add a README.md file with project description
echo "# My First Project" > README.md
echo "This is a sample project created to learn Git." >> README.md

# Stage all files
git add .

# Create the initial commit
git commit -m "Initial commit: Set up project structure"

# Output will look like:
# [main (root-commit) alb2c3d] Initial commit: Set up project structure
# 5 files changed, 10 insertions(+)
```

Chapter 3: CRUD Operations on Git

CRUD stands for Create, Read, Update, Delete. In Git's context, these operations map to tracking new files, reading history, updating tracked content, and removing files.

3.1 CREATE - Adding New Files (git add)

git add moves changes from your Working Directory to the Staging Area. It tells Git "I want to include these changes in my next snapshot (commit)."

```
# Stage a single specific file
git add index.html

# Stage multiple specific files
git add styles.css app.js

# Stage ALL changed and new files in the current directory and subdirectories
git add .

# Stage all changes across the entire repository (from anywhere)
git add -A

# Stage only MODIFIED files (not new untracked files)
git add -u

# Interactively choose which parts of a file to stage (useful for partial commits)
git add -p index.html
```

Note: git add . stages everything in the current folder and below. git add -A stages everything in the entire repository regardless of your current folder location.

Practical Example: Adding Files

```
# Imagine you modified index.html and created a new file contact.html

# Check what's changed first
git status
# Output:
# Changes not staged for commit:
#   modified:   src/index.html
# Untracked files:
#   src/contact.html

# Stage just the modified file
git add src/index.html
```

```
# Now stage the new untracked file
git add src/contact.html

# Check status again - both are now in staging
git status
# Output:
# Changes to be committed:
#   modified:   src/index.html
#   new file:   src/contact.html
```

3.2 CREATE - Saving Changes (git commit)

git commit takes everything in the Staging Area and saves it as a permanent snapshot in the repository history. Each commit has a unique SHA hash identifier, an author, timestamp, and message.

```
# Basic commit with an inline message
git commit -m "Add navigation menu to homepage"

# Commit with a multi-line message (opens your configured text editor)
git commit

# Skip staging - directly commit all tracked modified files (NOT new untracked files)
git commit -a -m "Fix typo in contact page"

# Amend the very last commit (change message or add forgotten files)
# WARNING: Do not amend commits already pushed to a shared remote
git commit --amend -m "Corrected: Add navigation menu to homepage"
```

Writing Good Commit Messages

A good commit message is crucial for team collaboration and future maintenance. Follow this convention:

```
# Format: <type>: <short summary (max 50 chars)>
#           [blank line]
#           [optional detailed description]

# Good examples:
git commit -m "feat: Add user registration form with validation"
git commit -m "fix: Resolve null pointer in login controller"
git commit -m "docs: Update README with setup instructions"
git commit -m "style: Format code according to ESLint rules"
git commit -m "refactor: Extract payment logic into separate service"
git commit -m "test: Add unit tests for user authentication"
```

```
# Bad examples (avoid these):
git commit -m "fix"           # Too vague
git commit -m "changes"       # Meaningless
git commit -m "WIP"           # Not descriptive
git commit -m "asdfgh"        # Not descriptive
```

3.3 READ - Checking Status (git status)

git status shows the current state of your working directory and staging area. It tells you which files are modified, which are staged, and which are untracked. Run this often!

```
# Show the full status
git status

# Example output and what each section means:
# On branch main
# Your branch is up to date with 'origin/main'.
#
# Changes to be committed:      <- These ARE staged (git add was used)
#   (use "git restore --staged <file>..." to unstage)
#       modified:   src/index.html
#       new file:   src/about.html
#
# Changes not staged for commit:  <- Modified but NOT yet staged
#   (use "git add <file>..." to update what will be committed)
#       modified:   src/styles.css
#
# Untracked files:               <- Brand new files Git does not know about
#   (use "git add <file>..." to include in what will be committed)
#       src/contact.html

# Short/compact status view
git status -s
# M  src/index.html  (M = modified, first column = staged, second = unstaged)
# ?? src/contact.html  (?? = untracked)
```

3.4 READ - Viewing Commit History (git log)

git log shows the complete commit history of the current branch. It is invaluable for understanding what changed, when, and by whom.

```
# Show full commit history
git log

# Show condensed one-line summary per commit
git log --oneline
```

```
# Show last 5 commits only
git log -5

# Show a visual graph of branches and merges
git log --oneline --graph --all

# Show commits by a specific author
git log --author="Jane Smith"

# Show commits after a specific date
git log --after="2024-01-01"

# Show commits that changed a specific file
git log -- src/index.html

# Show what actually changed in each commit (the diff)
git log -p

# Custom pretty format: hash, author, date, message
git log --pretty=format:"%h - %an, %ar : %s"
```

Reading git log Output

```
# Example output of 'git log':
commit a1b2c3d4e5f6g7h8i9j0k1l2m3n4o5p6q7r8s9t    <- Full SHA hash
Author: Jane Smith <jane@example.com>                <- Who made the commit
Date:   Mon Jan 15 14:30:00 2024 +0530                <- When it was committed

    feat: Add user registration form                  <- The commit message

    Added HTML form, CSS styling, and JavaScript
    validation for the new user registration page.

commit b2c3d4e5f6g7h8i9j0k1l2m3n4o5p6q7r8s9t0u    <- Previous commit
Author: John Doe <john@example.com>
Date:   Sun Jan 14 10:15:00 2024 +0530

    Initial commit: Set up project structure
```

3.5 UPDATE - Pushing Changes to Remote (git push)

git push uploads your local commits to a remote repository (like GitHub). This makes your work visible to collaborators and backs it up on the server.

```
# First, connect your local repo to a remote (one-time setup)
git remote add origin https://github.com/yourusername/my-project.git
```

```
# Verify remotes are set up
git remote -v
# Output:
# origin  https://github.com/yourusername/my-project.git (fetch)
# origin  https://github.com/yourusername/my-project.git (push)

# Push the main branch to the remote (first time, set upstream tracking)
git push -u origin main

# After the first push, you can simply use:
git push

# Push a specific branch to remote
git push origin feature/login-page

# Push all local branches to remote
git push --all origin

# Force push (DANGEROUS - overwrites remote history, use only on personal
branches)
git push --force origin feature/login-page
```

3.6 UPDATE - Pulling Changes from Remote (git pull)

git pull downloads commits from the remote repository and immediately merges them into your current local branch. It is essentially git fetch + git merge in one command.

```
# Pull and merge latest changes from origin into current branch
git pull

# Explicitly specify remote and branch
git pull origin main

# Pull using rebase instead of merge (creates cleaner linear history)
git pull --rebase origin main

# Fetch without merging (see what's new before deciding to merge)
git fetch origin
git log HEAD..origin/main --oneline # Compare local vs remote
git merge origin/main             # Then merge when ready
```

3.7 UPDATE - Merging Branches (git merge)

git merge combines changes from one branch into another. When you finish working on a feature branch, you merge it back into main.

```
# Step 1: Switch to the branch you want to merge INTO
git checkout main

# Step 2: Make sure main is up to date
git pull origin main

# Step 3: Merge the feature branch
git merge feature/user-registration

# If there are no conflicts, Git creates a merge commit automatically.
# Output example:
# Updating alb2c3d..e4f5g6h
# Fast-forward
#  src/register.html | 45 ++++++
#  src/register.js   | 78 ++++++++
#  2 files changed, 123 insertions(+)
```

Handling Merge Conflicts

A merge conflict happens when two branches modify the same line(s) in a file. Git cannot automatically decide which change to keep, so you must resolve it manually.

```
# After running git merge, if there's a conflict:
# CONFLICT (content): Merge conflict in src/index.html
# Automatic merge failed; fix conflicts and then commit the result.

# Open the conflicting file - it will contain conflict markers:
<<<<<<< HEAD
<title>My Website - Version A</title>
=====
<title>My Website - New Title</title>
>>>>>>> feature/update-title

# Explanation:
# <<<<<<< HEAD           = Start of YOUR changes (current branch)
# =====               = Separator between the two versions
# >>>>>>> branch-name    = Start of THEIR changes (branch being merged)

# To resolve: manually edit the file to keep the correct version:
<title>My Website - New Title</title>
# (delete all the <<<, ==, >>> markers too)

# Stage the resolved file and complete the merge
git add src/index.html
git commit -m "Merge feature/update-title: use new page title"
```

3.8 DELETE - Removing Files from Git

```
# Remove a file from both the working directory AND Git tracking
git rm src/oldfile.html
git commit -m "Remove deprecated oldfile.html"

# Remove a file from Git tracking ONLY (keep it on disk)
# Useful when you forgot to add a file to .gitignore
git rm --cached config/secrets.env
echo "config/secrets.env" >> .gitignore
git commit -m "Stop tracking secrets.env, add to .gitignore"

# Remove an entire folder from tracking
git rm -r --cached node_modules/

# Undo staged changes (unstage a file, move back to working directory)
git restore --staged src/index.html

# Discard all unstaged changes to a file (PERMANENT - cannot undo!)
git restore src/index.html

# Reset to a previous commit (--soft keeps changes staged)
git reset --soft HEAD~1

# Reset to a previous commit (--hard DISCARDS all changes - very dangerous!)
git reset --hard HEAD~1
```


Chapter 4: Overview of GitHub

4.1 What is GitHub?

GitHub is a cloud-based hosting platform for Git repositories. While Git is the version control tool you use locally on your computer, GitHub is a website and service that stores your Git repositories online. GitHub adds collaboration features on top of Git including:

- A web-based graphical interface to browse and manage repositories
- Pull Requests for code review and discussion before merging
- Issues for bug tracking, feature requests, and project management
- GitHub Actions for continuous integration and deployment (CI/CD)
- GitHub Pages for hosting static websites directly from a repository
- A large social coding community with millions of open-source projects

GitHub vs Git - Key Distinction

Git	GitHub
A software tool (version control system)	A web platform (hosting service)
Runs on your local computer	Lives in the cloud (internet)
Works completely offline	Requires internet connection
Created by Linus Torvalds (2005)	Founded by Tom Preston-Werner (2008)
Free and open source	Free tier + paid plans available
Command-line focused	Web UI + command-line

4.2 GitHub Repositories

A GitHub repository (repo) is the online counterpart to your local Git repository. It stores all your project files, commit history, branches, and issues in one place accessible from anywhere.

Creating a New Repository on GitHub

5. Log in to github.com and click the "+" button in the top-right corner
6. Select "New repository"
7. Fill in: Repository name, Description (optional), Visibility (Public or Private)
8. Optionally initialize with a README, .gitignore, and license
9. Click "Create repository"

```
# After creating the repo on GitHub, connect your local repo to it:

# Option A: If you have an existing local repo
git remote add origin https://github.com/yourusername/repo-name.git
git branch -M main
git push -u origin main

# Option B: Clone the newly created GitHub repo to your local machine
git clone https://github.com/yourusername/repo-name.git
cd repo-name
```

4.3 GitHub Branches

GitHub branches work the same as Git branches but are visible and manageable through the GitHub web interface. GitHub shows all remote branches and allows branch management through its UI.

- The default branch (usually main) is the production-ready code
- Feature branches allow isolated development and are visible to all collaborators
- Branch protection rules can prevent direct pushes to main, requiring pull requests
- You can create, rename, and delete branches directly on GitHub's website

```
# View all branches (local and remote)
git branch -a

# Push a local branch to GitHub to make it visible online
git push origin feature/new-dashboard

# Pull a remote branch that exists on GitHub but not locally
git checkout -b feature/new-dashboard origin/feature/new-dashboard

# Delete a remote branch on GitHub from command line
git push origin --delete feature/old-feature
```

4.4 GitHub Issues

Issues are GitHub's built-in bug tracker and task manager. They are used to track bugs, propose new features, ask questions, and manage project tasks. Every issue gets a unique number (#1, #2, etc.).

Creating and Managing Issues

10. Go to your repository on GitHub
11. Click the "Issues" tab
12. Click "New issue"
13. Add a descriptive title and detailed description
14. Assign labels (bug, enhancement, documentation, etc.)

15. Assign to a team member, add to a milestone or project board
16. Click "Submit new issue"

Note: You can reference issues in commit messages using # + issue number. For example: `git commit -m "fix: Resolve login timeout issue, closes #42"` - GitHub will automatically close issue #42 when this commit is merged to main.

Useful Issue Keywords

Keyword in commit/PR	Effect when merged to main
closes #42	Automatically closes issue #42
fixes #15	Automatically closes issue #15
resolves #7	Automatically closes issue #7
relates to #20	Links the commit to issue #20 (does not close it)

4.5 Pull Requests (PRs)

A Pull Request (PR) is a GitHub feature that lets you notify team members that you have pushed code to a branch and it is ready to be reviewed and merged into another branch (usually main). PRs are the heart of collaborative development on GitHub.

Creating a Pull Request

17. Push your feature branch to GitHub: `git push origin feature/login-page`
18. Go to your repository on GitHub
19. GitHub will usually show a prompt: "Compare & pull request" - click it
20. Or click "Pull requests" tab, then "New pull request"
21. Select the base branch (e.g., main) and compare branch (your feature branch)
22. Write a descriptive title and description of your changes
23. Assign reviewers, add labels, link to issues
24. Click "Create pull request"

The Pull Request Review Process

```
# Typical PR workflow in a team:

1. Developer creates a feature branch and pushes code
  git checkout -b feature/payment-gateway
  # ... make changes ...
  git push origin feature/payment-gateway

2. Developer opens a Pull Request on GitHub
  - Describes what was changed and why
  - Links to related issues
```

3. Team lead/peer reviews the code on GitHub
 - Comments on specific lines
 - Requests changes or approves
4. Developer addresses feedback
 - # Make requested changes locally
 - `git add .`
 - `git commit -m "fix: Address PR review feedback - improve error handling"`
 - `git push origin feature/payment-gateway`
 - # The PR updates automatically with new commits
5. Once approved, PR is merged into main
 - On GitHub: click 'Merge pull request'
 - Or via command line:
 - `git checkout main`
 - `git merge feature/payment-gateway`
 - `git push origin main`
6. Feature branch is deleted
 - `git branch -d feature/payment-gateway`
 - `git push origin --delete feature/payment-gateway`

4.6 GitHub Collaboration Features

GitHub offers powerful tools for team collaboration beyond just code storage:

Forking

Forking creates your own copy of someone else's repository under your GitHub account. This is commonly used in open-source contribution workflows.

```
# After forking on GitHub, clone your fork locally
git clone https://github.com/YOUR-USERNAME/original-project.git

# Add the original repository as 'upstream' to track updates
git remote add upstream https://github.com/ORIGINAL-OWNER/original-project.git

# Keep your fork up to date with the original
git fetch upstream
git checkout main
git merge upstream/main
git push origin main
```

GitHub Project Boards & Wikis

- **Project Boards:** Kanban-style boards to organize and prioritize issues and pull requests into columns like To Do, In Progress, Done

- Wikis: Built-in documentation pages for your project, great for onboarding docs, architecture decisions, API references
- Discussions: A forum-like space for open-ended conversations, Q&A, and announcements separate from Issues
- GitHub Actions: Automated workflows that run tests, builds, and deployments whenever you push code

Chapter 5: Set Up and Configure GitHub Account

5.1 Creating a GitHub Account

25. Visit <https://github.com> in your web browser
26. Click "Sign up" and enter your email address
27. Create a strong password and choose a unique username (this will be visible publicly)
28. Verify your email address by clicking the link sent to your inbox
29. Complete the setup wizard and choose the Free plan

Note: Choose your GitHub username carefully. It appears in all your repository URLs (github.com/yourusername/repo-name) and will be seen by potential employers and collaborators.

5.2 Setting Up SSH Keys

SSH (Secure Shell) keys provide a secure way to authenticate with GitHub without typing your username and password every time. An SSH key pair consists of a private key (stays on your machine) and a public key (added to GitHub).

Step 1: Check for Existing SSH Keys

```
# List any existing SSH keys in the default directory
ls -al ~/.ssh

# Look for files named:
# id_rsa and id_rsa.pub (RSA algorithm - older)
# id_ed25519 and id_ed25519.pub (Ed25519 - modern, recommended)
```

Step 2: Generate a New SSH Key

```
# Generate a new Ed25519 SSH key (recommended algorithm)
ssh-keygen -t ed25519 -C "your.email@example.com"

# If your system doesn't support Ed25519, use RSA:
ssh-keygen -t rsa -b 4096 -C "your.email@example.com"

# When prompted:
# Enter file in which to save the key: press ENTER for default location
# Enter passphrase: type a secure passphrase (or press ENTER for no passphrase)
# Enter same passphrase again: confirm your passphrase

# Output will show something like:
# Your identification has been saved in /home/user/.ssh/id_ed25519
# Your public key has been saved in /home/user/.ssh/id_ed25519.pub
```

Step 3: Add SSH Key to the SSH Agent

```
# Start the SSH agent in the background
eval "$(ssh-agent -s)"
# Output: Agent pid 12345

# Add your private key to the agent
ssh-add ~/.ssh/id_ed25519

# On macOS, add to Keychain for persistence across restarts:
ssh-add --apple-use-keychain ~/.ssh/id_ed25519
```

Step 4: Add SSH Public Key to GitHub

```
# Copy the PUBLIC key content to your clipboard

# macOS:
pbcopy < ~/.ssh/id_ed25519.pub

# Linux (requires xclip):
xclip -selection clipboard < ~/.ssh/id_ed25519.pub

# Windows (Git Bash):
cat ~/.ssh/id_ed25519.pub | clip

# Or simply display it and manually copy:
cat ~/.ssh/id_ed25519.pub

# Then on GitHub:
# 1. Go to Settings (click your profile picture, top right)
# 2. Click 'SSH and GPG keys'
# 3. Click 'New SSH key'
# 4. Add a title (e.g., 'My Work Laptop')
# 5. Paste the public key content
# 6. Click 'Add SSH key'
```

Step 5: Test Your SSH Connection

```
# Test the SSH connection to GitHub
ssh -T git@github.com

# Expected successful output:
# Hi yourusername! You've successfully authenticated, but GitHub
# does not provide shell access.

# Now use SSH URLs instead of HTTPS for cloning:
git clone git@github.com:yourusername/repository-name.git
```

```
# Change an existing remote from HTTPS to SSH:  
git remote set-url origin git@github.com:yourusername/repository-name.git
```

5.3 Personal Access Tokens (PATs)

Personal Access Tokens (PATs) are an alternative to SSH keys for authentication. They are especially useful for HTTPS-based authentication, scripts, and third-party integrations. Since August 2021, GitHub no longer accepts your account password for Git operations via HTTPS - you must use a PAT instead.

Creating a Personal Access Token

30. Log in to GitHub and click your profile picture (top right)
31. Go to Settings > Developer settings > Personal access tokens > Tokens (classic)
32. Click "Generate new token" then "Generate new token (classic)"
33. Give it a descriptive name (e.g., "My Dev Laptop - Git Operations")
34. Set an expiration date (90 days is a good balance of security and convenience)
35. Select the required scopes/permissions:
 - repo - Full control of private repositories (required for most Git operations)
 - workflow - Update GitHub Actions workflows (if needed)
 - read:org - Read organization data (if working in an organization)
36. Click "Generate token" and IMMEDIATELY copy the token - you will never see it again

```
# Using your PAT for HTTPS authentication:  
# When Git asks for your password, paste the PAT instead of your account password  
  
# Store credentials to avoid re-entering the token every time:  
  
# macOS (uses Keychain):  
git config --global credential.helper osxkeychain  
  
# Windows:  
git config --global credential.helper manager  
  
# Linux (stores in memory for 15 minutes):  
git config --global credential.helper cache  
  
# Linux (stores permanently on disk - less secure):  
git config --global credential.helper store  
  
# After configuring, your next push will ask for credentials.  
# Enter your GitHub username and paste your PAT as the password.  
# It will be stored automatically afterwards.
```

Token Types: Classic vs Fine-grained

Classic Tokens	Fine-grained Tokens (Recommended)
Broad permissions (repo, user, org level)	Granular permissions per repository
Apply to all your repositories	Restrict access to specific repos only
Simpler to set up	More secure for production/automation use
Good for personal development use	Best for CI/CD pipelines and scripts

5.4 Repository Permissions and Access Control

GitHub allows fine-grained control over who can access and what they can do in your repositories.

Repository Visibility

Visibility	Description
Public	Anyone on the internet can see the repository. Only collaborators can push.
Private	Only you and explicitly invited collaborators can see and access the repo.
Internal (Enterprise only)	Visible to all members of your GitHub organization, not the public.

Collaborator Permission Levels

Role	Permissions
Read	Can view and clone the repository. Can open issues and comment on PRs.
Triage	Read + can manage issues and pull requests without write access to code.
Write	Triage + can push to branches, create branches, merge approved PRs.
Maintain	Write + can manage repo settings (not destructive settings).
Admin	Full access including destructive actions: delete repo, manage team access.

Adding a Collaborator to Your Repository

37. Go to your repository on GitHub
38. Click "Settings" tab
39. Click "Collaborators" in the left sidebar
40. Click "Add people"
41. Search by GitHub username or email address
42. Select their permission level

43. Click "Add [username] to this repository"
44. The collaborator receives an email invitation they must accept

Setting Up Branch Protection Rules

Branch protection rules prevent accidental direct pushes to critical branches (like main) and enforce workflows like code review.

45. Go to repository Settings > Branches
46. Click "Add branch protection rule"
47. Enter the branch name pattern (e.g., main)
48. Configure protections:
 - Require pull request reviews before merging (recommended: 1-2 approvals)
 - Require status checks to pass before merging (CI tests must pass)
 - Require branches to be up to date before merging
 - Restrict who can push directly to this branch
 - Do not allow force pushes (protects commit history)
49. Click "Create" to save the protection rule

Quick Reference: Essential Git & GitHub Commands

6.1 Most Used Git Commands

Command	What it Does
git init	Initialize a new empty Git repository in the current folder
git clone <url>	Download a remote repository to your local machine
git status	Show the state of your working directory and staging area
git add <file>	Stage a specific file for the next commit
git add .	Stage ALL changes in the current directory
git commit -m "msg"	Save staged changes as a new commit with a message
git log --oneline	View condensed commit history
git branch	List all local branches
git checkout -b <name>	Create and switch to a new branch
git merge <branch>	Merge a branch into the current branch
git rebase <branch>	Rebase current branch onto another branch
git push origin <branch>	Push local commits to GitHub
git pull origin <branch>	Fetch and merge remote changes
git remote -v	List configured remote repositories
git stash	Temporarily save uncommitted changes
git stash pop	Restore the most recently stashed changes
git diff	Show unstaged changes in files
git reset --soft HEAD~1	Undo last commit, keep changes staged
git restore <file>	Discard unstaged changes to a file

6.2 Everyday Workflow Summary

```
# --- DAILY WORKFLOW ---

# 1. Start your day: get latest changes from remote
git pull origin main

# 2. Create a new branch for your task
git checkout -b feature/your-task-name

# 3. Make your code changes...
```

```
# 4. Check what you changed
git status
git diff

# 5. Stage your changes
git add .

# 6. Commit with a meaningful message
git commit -m "feat: Describe what you did clearly"

# 7. Push your branch to GitHub
git push origin feature/your-task-name

# 8. On GitHub: open a Pull Request for code review

# 9. After PR is approved and merged, clean up
git checkout main
git pull origin main
git branch -d feature/your-task-name
```

Note: Practice makes perfect! The best way to learn Git is to use it every day. Create a personal project on GitHub and commit your code regularly. Over time, these commands will become second nature.

End of Tutorial

Happy Coding!